

Quest 부팅 시 추나 앱 자동실행

— 문제해결 전 과정 정리

GuideChuna (VR 추나요법 교육 앱) · Quest 3 / 3S (Horizon OS)

작성: 2026-06-16 · 상태: **실기기 검증 완료**

1. 배경과 목표

목표: Quest 전원을 켜면 별도 조작 없이 추나 앱이 자동으로 실행·진입되게 한다.

왜 필요한가: 교육 현장에서 학습자(주로 비숙련 사용자)가 헤드셋을 켜 뒤 **메타(Meta) 셀 메뉴에서 앱을 직접 찾아 실행하는 단계에서 막히는** 문제가 반복됐다. 부팅 직후 앱이 바로 떠 있으면 이 진입 장벽이 사라진다.

2. 정식 해법이 막힌 이유

부팅 자동실행의 **정식 경로**는 **키오스크 모드(MDM)**이다 — Meta for Work의 HMS(Headset Management Service) 또는 ArborXR 같은 MDM 솔루션으로 디바이스를 등록하면 OS 차원에서 자동실행·셀 억제가 보장된다.

그러나:

- 한국이 **Meta for Work** 지원 국가에서 빠져 있어 셀프서비스 가입이 막힌다.
- 우회로(미국 조직 생성 / ArborXR 무료 50대 지원 문의)는 운영·계정 비용이 크다.

→ 계정·디바이스 등록이 **불필요한 DIY(자체) 부트 리시버** 방식을 채택했다. 단, 이 방식은 본질적으로 OS 보증이 없어 타이밍·리소스·시스템 다이얼로그에 취약하다는 점을 전제로 시작했다.

3. 핵심 구조

```
[전원 ON]
  |
  ▼
BootReceiver.java —(BOOT_COMPLETED / QUICKBOOT_POWERON 수신)
  |
  └─ onReceive: 즉시 리턴 (브로드캐스트 ANR 방지)
  |
  ▼
AlarmManager —(+15초 후 1회 발화)
  |
  ▼
getLaunchIntentForPackage(self) → startActivity
  |   ※ SYSTEM_ALERT_WINDOW(SAW) 권한으로 BAL 면제
  |
  ▼
[추나 앱 최상위 진입]
```

- `Assets/Plugins/Android/BootReceiver.java` (신규): `BOOT_COMPLETED` / `QUICKBOOT_POWERON` 수신 → `getLaunchIntentForPackage(getPackageName())` 로 본 앱 실행. 패키지명을 하드코딩하지 않아 메인 빌드와 `.handrecord` 빌드가 공용으로 쓴다. Unity 6 Gradle 빌드가 `Plugins/Android/*.java` 를 자동 컴파일한다.
- `AlarmManager`로 부팅 +15초 후 단일 실행 (`LAUNCH_DELAY_MS = 15000`). `onReceive` 는 알람만 예약하고 즉시 리턴하므로 브로드캐스트 ANR이 없다. 늦게 띄워 부팅 직후 리소스 폭주를 피한다.
- `SYSTEM_ALERT_WINDOW` (SAW) 권한으로 BAL(Background Activity Launch) 면제 → 면제창 밖(+15초) 실행이 차단되지 않게 한다.

4. 시행착오 — 이 순서로 좁혀졌다

문제의 본질은 세 가지 제약이 서로 충돌하는 3중 딜레마였다.

#	시도	결과	원인
1	즉시 실행	앱은 뜨나 메타 셸이 직후 올라와 덮음	포커스 레이스
2	goAsync 8.5초 + 5회 재 시도	앱은 최상위로 가나 브로드캐스트 ANR + 텅김	장시간 점유 + 반복 실행이 Unity 초기화와 충돌
3	AlarmManager +12초 단일 실행	BAL_BLOCK으로 안 뜸	부트 면제창 밖
4	goAsync 4초 단일 실행	한 번은 성공했으나 반복 재부팅 시 ANR → 강제종료	4초는 너무 일러 리소스 폭주 (lowmemorykiller-vrruntime 크래시) 중 무거운 Unity가 못 버텨
5	(최종 A안) SAW 면제 + AlarmManager 15초	성공	ANR·BAL 동시 회피

3중 딜레마의 정체

- 일찍 실행(4초): 부팅 리소스 폭주 구간에 무거운 Unity가 들어가 `ANR(Input dispatch timeout, FocusEvent 500ms)` → 강제종료.
- goAsync 오래 점유(8.5초): 브로드캐스트 자체가 ANR.
- AlarmManager 늦게(+12초): 부트 면제창 밖이라 `BAL_BLOCK`.

→ "리소스가 안정된 뒤(15초) + BAL을 면제(SAW)" 두 조건을 동시에 만족시켜야 빠져나올 수 있었다.

5. BAL 진단 — 가장 중요한 기술적 발견

실기기 `logcat` 으로 규명한, 통념과 반대되는 사실:

- 부팅이라고 백그라운드 액티비티 실행 권한이 자동으로 붙지 않는다. 부트 면제창 안(4초)이어도 리시버의 직접 `startActivity` 는 `BackgroundStartPrivileges[allowsBackgroundActivityStarts=false]` 로 `BAL_BLOCK` 된다 (`callingUidProcState: RECEIVER`).

- 그런데 Quest VR 셸(`ShellControlBroadcastReceiver`)이 우리 launch를 감지해 `com.oculus.vrshell.intent.action.LAUNCH` 로 `PendingIntent` 재실행 → `BAL_ALLOW_PENDING_INTENT` 로 성공시키는 경우가 있었다. 단 이 셸 구제는 부팅 직후 "앱 실행" 단계의 launch만 가로채 재발행하는 듯했고, 들쭉날쭉했다.
- **AlarmManager(+12초)가 안 뜬 이유:** 면제창 밖 + 이 셸 구제도 못 받음 → `BAL_BLOCK`.

해결책: 셸의 불안정한 구제에 의존하지 않고, `SYSTEM_ALERT_WINDOW` 권한으로 BAL을 직접 면제받는다. 그러면 +15초(면제창 밖) 실행도 `BAL_ALLOW_SAW_PERMISSION` 으로 통과한다.

SAW의 함정: manifest에 권한을 선언만 해서는 appop이 "default" 라 실제로 안 먹는다. 기기마다 `adb shell appops set com.CoCo.CHUNA SYSTEM_ALERT_WINDOW allow` 를 1회 부여해야 한다(재부팅 후에도 유지). 이것이 A안의 운영 비용이다.

6. 숨은 실패 경로 수정 — SCHEDULE_EXACT_ALARM (2026-06-15)

실기기 검증 직전 코드 점검에서 발견한 silent-failure:

- `AndroidTargetSdkVersion: 32` (API 31+)이라 `setExactAndAllowWhileIdle()` 호출이 `SCHEDULE_EXACT_ALARM` 권한을 요구하는데, manifest 주입 목록에 그 권한이 빠져 있었다.
- 권한 없이 호출 시 `SecurityException` → `scheduleDelayedLaunch` 의 try/catch에 삼켜져 **알람이 조용히 안 걸리고 자동실행이 통째로 실패**한다. SAW(BAL 면제)가 멀쩡해도 그 앞단 알람 예약에서 죽는 구조라, 검증해도 "안 뚫"만 나오고 원인 추적이 어려웠을 함정이었다.

수정:

- **BootReceiver.java** : 알람 예약 분기 재작성. API 31+(`VERSION_CODES.S`)에서 `alarmManager.canScheduleExactAlarms()` 로 가능 여부 확인 → 가능하면 `setExactAndAllowWhileIdle` (exact), 불가하면 `setAndAllowWhileIdle` (inexact, 권한 불필요)로 폴백. 부팅 +15초는 밀리초 정밀도가 불필요하므로 inexact로 충분하다. pre-M 경로는 기존 `setExact` 유지.
- **KioskAutoLaunchBuild.cs** : `android.permission.SCHEDULE_EXACT_ALARM` 을 키오스크 manifest 주입 목록에 추가(BOOT/SAW 다음). 맥등 처리.

이 inexact 폴백이 검증 성공의 전제였음이 뒤에 실증된다(아래 §8).

7. 스토어 리젝 방어 — 자동 게이팅

이 앱은 권한 문제로 v1.25~28 스토어 리젝 이력이 있어, 부팅 자동실행을 메인 빌드에 그대로 넣으면 정책 리젝 위험이 있다.

- 정적 `AndroidManifest.xml` 은 클린(부트 항목 0)으로 유지한다.
- **Assets/Editor/KioskAutoLaunchBuild.cs** (`IPostGenerateGradleAndroidProject`)가 **Scripting Define** `CHUNA_KIOSK_AUTOLAUNCH` 가 ON인 빌드에서만 생성된 manifest에 `RECEIVE_BOOT_COMPLETED` + `SYSTEM_ALERT_WINDOW` + `SCHEDULE_EXACT_ALARM` 권한과 리시버 XML을 주입한다. 맥등(이미 있으면 skip).
- 메뉴: **Build > 키오스크 자동실행 ON/OFF/상태 확인**.

- `BootReceiver.java` 는 항상 컴파일되지만 manifest 미등록 시 호출 안 되는 죽은 코드다(정책 스캔은 manifest 기준이라 무해).

주의: define 토글 후에는 **재컴파일**이 끝난 다음 빌드해야 주입된다. 토글 직후 바로 빌드하면 stale 어셈블리로 누락된다.

8. 실기기 검증 결과 (2026-06-15)

환경: Quest 3S / Horizon OS / **Android 14 (API 34)**

8-1. 단일 부팅 성공

성공 로그 시퀀스 (adb logcat / dumpsys):

```
Start proc CHUNA for broadcast BootReceiver          (+0s)
START CHUNA/AppUIGameActivity (BAL_ALLOW_SAW_PERMISSION) result code=0  (+15s)
WindowManager Transition OPEN CHUNA
topResumedActivity = com.CoCo.CHUNA/AppUIGameActivity  (최상위 안착)
```

- **BAL이 SAW 권한으로 허용됨**(`BAL_ALLOW_SAW_PERMISSION`) = 설계한 면제 메커니즘이 정확히 작동. `BAL_BLOCK` 0건.
- 기기가 **API 34였음** → `SCHEDULE_EXACT_ALARM` granted = **false**(API 33+는 선언만으론 자동허용 안 됨). 즉 §6에서 넣은 **inexact** 폴백이 없었으면 이 기기에서 **알람 자체가 안 걸렸을 것** — 수정이 검증의 전제였음이 실증됐다.

8-2. 반복 안정성 (연속 5회 재부팅)

- **실행 메커니즘은 5/5 (100%) 재현.** 예전 4초 goAsync판의 "들쭉날쭉"이 SAW+15초 AlarmManager로 완전히 해소됐다.
- 그러나 콜드부팅 직후 화면 최상위가 추나가 아니었다 — `topResumedActivity` 가 부팅 +18~48초 구간 내내 `com.oculus.guardian/GuardianDialogActivity` (경계선 확인 다이얼로그)였다. 추나는 그 뒤에서 정상 실행·생존하지만 Guardian이 앞을 막았다.

8-3. Guardian이 유일한 잔여 차단 요인임을 격리

- **인과 격리 증명:** `setprop debug.oculus.guardian_pause 1` + `am force-stop com.oculus.guardian` + 추나 재실행 → 즉시 `topResumedActivity = com.CoCo.CHUNA`. **Guardian만 제거하면 추나가 최상위가 됨.** → 부팅 후 "추나 안 보임"의 단일 원인 = Guardian 다이얼로그로 확정.
- **한계:** `guardian_pause` 는 setprop 런타임값이라 재부팅 시 초기화(persist 아님)된다. Guardian 영속 비활성은 adb로 불가하고, **헤드셋 내 개발자 설정의 경계선(Guardian) 토글 OFF**(영속, 기기별 1회) 또는 HMS/MDM 키오스크에서만 가능하다.

8-4. 최종 확정 — 경계선 OFF 상태 재부팅

- 경계선(Guardian)을 OFF한 상태로 재부팅 후 `topResumedActivity` 를 8초 간격 12회(부팅 +0s ~ +88s) 연속 추적 → 전 구간 `com.CoCo.CHUNA/AppUIGameActivity` 유지, **셀에 한 번도 안 뺏김**. 프로세스 생존.

- 결론: 경계선 OFF가 적용되면 추나가 부팅 직후부터 계속 최상위로 안착 = 부팅 자동실행 기능 전체(실행 + 가시성) 검증 완료.

9. 운영 체크리스트 (기기별 배포 절차)

부팅 직후 사용자가 추나를 바로 보려면 기기마다 다음 선행 세팅이 필요하다.

1. 키오스크 define ON 빌드 — Build > 키오스크 자동실행 ON → 재컴파일 후 빌드.
2. 설치 후 앱 1회 수동 실행 — APK 재설치 직후 앱은 `stopped=true` 라 `BOOT_COMPLETED` 가 미전달된다. 1회 실행해야 `stopped=false` 로 풀려 부팅 브로드캐스트를 받는다. (재부팅만으론 안 풀림. 플래시 직후 첫 부팅은 자동실행 안 됨 — 안드로이드 본질 제약.)
3. SAW appop 부여 — `adb shell appops set com.CoCo.CHUNA SYSTEM_ALERT_WINDOW allow` (BAL 면제용, 기기별 1회, 재부팅 유지).
4. 헤드셋 개발자 설정에서 경계선/Guardian OFF (영속, 기기별 1회).

→ 이후 콜드부팅마다 ~15초 뒤 추나가 자동 진입·최상위.

adb 검증 명령은 Unity 번들 adb 사용:

`C:\Users\USER\AppData\Local\Android\Sdk\platform-tools\adb.exe`

10. 한계와 향후

- 메타 셀 자체는 억제하지 못한다. 메타 버튼 메뉴 소환은 여전히 유효하고, 경계선이 켜져 있으면 부팅마다 Guardian 다이얼로그가 뜬다(추나는 이미 뒤에서 실행 중이라 1회 통과하면 됨).
- SAW grant-Guardian OFF가 기기별 수작업이다. 플릿 배포 시 헤드셋마다 필요 → grant 자동화 스크립트 검토 여지.
- 무거운 VR 앱을 부팅 환경에 직접 던지는 구조라 본질적으로 취약하다(타이밍·리소스·Guardian 의존). 완전한 견고함은 키오스크(HMS/MDM)뿐 — A안이 흔들리면 B(HMS) 풀백.

부록 A. 관련 파일

파일	역할
<code>Assets/Plugins/Android/BootReceiver.java</code>	부트 수신 + AlarmManager 예약 + 앱 실행
<code>Assets/Editor/KioskAutoLaunchBuild.cs</code>	define ON 시 manifest 권한/리시버 주입 (게이팅)
<code>Assets/Plugins/Android/AndroidManifest.xml</code>	정적 클린 유지(부트 항목 없음)
<code>ProjectSettings/ProjectSettings.asset</code>	targetSdk / Scripting Define

부록 B. 용어

- BAL (Background Activity Launch): 백그라운드에서 액티비티를 띄우는 행위. Android 10+에서 기본 차단되며, 부팅 브로드캐스트 컨텍스트도 자동 허용되지 않는다.

- **SAW (SYSTEM_ALERT_WINDOW):** "다른 앱 위에 표시" 권한. 부여 시 BAL 면제 사유 (`BAL_ALLOW_SAW_PERMISSION`)가 된다.
- **appop:** 권한 선언과 별개로 OS가 실제 허용 여부를 관리하는 op 상태. SAW는 `default` 상태에선 동작하지 않아 명시적 `allow` 가 필요하다.
- **Guardian / 경계선:** Quest의 플레이 영역 안전 경계 시스템. 부팅 시 확인 다이얼로그를 띄워 최상위를 점유한다.
- **stopped 상태:** 설치/강제종료 후 앱이 한 번도 실행되지 않은 상태. 이 상태에선 브로드캐스트가 전달되지 않는다.